RESEARCH-ARTICLE

On the Effectiveness of Large Language Models in Domain-Specific Code Generation

XIAODONG GU, Shanghai Jiao Tong University, Shanghai, China

MENG CHEN, Shanghai Jiao Tong University, Shanghai, China

YALAN LIN, Shanghai Jiao Tong University, Shanghai, China

YUHAN HU, Shanghai Jiao Tong University, Shanghai, China

HONGYU ZHANG, Chongqing University, Chongqing, China

CHENGCHENG WAN, East China Normal University, Shanghai, China

View all

Open Access Support provided by:

Tencent

Shanghai Jiao Tong University

Chongqing University

East China Normal University

Published: 23 February 2025
Online AM: 01 October 2024
Accepted: 19 August 2024
Revised: 04 July 2024
Received: 04 December 2023

On the Effectiveness of Large Language Models in Domain-Specific Code Generation

XIAODONG GU, MENG CHEN, YALAN LIN, and YUHAN HU, Shanghai Jiao Tong University, Shanghai, China
HONGYU ZHANG, Chongqing University, Chongqing, China
CHENGCHENG WAN, East China Normal University, Shanghai, China
ZHAO WEI, YONG XU, and JUHONG WANG, Tencent Inc., Beijing, China

Large language models (LLMs) such as ChatGPT have shown remarkable capabilities in code generation. Despite significant achievements, they rely on enormous training data to acquire a broad spectrum of open-domain knowledge. Besides, their evaluation revolves around open-domain benchmarks like HumanEval, which primarily consist of programming contests. Therefore, it is hard to fully characterize the intricacies and challenges associated with particular domains (e.g., Web, game, and math). In this article, we conduct an in-depth study of the LLMs in domain-specific code generation. Our results demonstrate that LLMs exhibit sub-optimal performance in generating domain-specific code, due to their limited proficiency in utilizing domain-specific libraries. We further observe that incorporating API knowledge as prompts can empower LLMs to generate more professional code. Based on these findings, we further investigate how to effectively incorporate API knowledge into the code generation process. We experiment with three strategies for incorporating domain knowledge, namely, external knowledge inquirer, chain-of-thought prompting, and chain-of-thought fine-tuning. We refer to these strategies as a new code generation approach called *DomCoder*. Experimental results show that all strategies of DomCoder improve the effectiveness of domain-specific code generation under certain settings.

CCS Concepts: • **Software and its engineering → Automatic programming**;

Additional Key Words and Phrases: large language models, code generation, domain-specific program generation

ACM Reference format:
Xiaodong Gu, Meng Chen, Yalan Lin, Yuhan Hu, Hongyu Zhang, Chengcheng Wan, Zhao Wei, Yong Xu, and Juhong Wang. 2025. On the Effectiveness of Large Language Models in Domain-Specific Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 3, Article 78 (February 2025), 22 pages.
https://doi.org/10.1145/3697012

This research is supported by the National Key R&D Program of China (Grant No. 2023YFB4503802), the National Natural Science Foundation of China (Grant No. 62102244), and the CCF-Tencent Open Research Fund (RAGR20220129).
Authors' Contact Information: Xiaodong Gu (corresponding author), Shanghai Jiao Tong University, Shanghai, China; e-mail: xiaodong.gu@sjtu.edu.cn; Meng Chen, Shanghai Jiao Tong University, Shanghai, China; e-mail: mengchen@sjtu.edu.cn; Yalan Lin, Shanghai Jiao Tong University, Shanghai, China; e-mail: linyalan@sjtu.edu.cn; Yuhan Hu, Shanghai Jiao Tong University, Shanghai, China; e-mail: suzhengv@sjtu.edu.cn; Hongyu Zhang, Chongqing University, Chongqing, China; e-mail: hyzhang@cqu.edu.cn; Chengcheng Wan, East China Normal University, Shanghai, China; e-mail: ccwan@sei.ecnu.edu.cn; Zhao Wei, Tencent Inc., Beijing, China; e-mail: zachwei@tencent.com; Yong Xu, Tencent Inc., Beijing, China; e-mail: rogerxu@tencent.com; Juhong Wang, Tencent Inc., Beijing, China; e-mail: julietwang@tencent.com.

ACM 1557-7392/2025/2-ART78
https://doi.org/10.1145/3697012

1 Introduction

Large language models (LLMs), such as ChatGPT, have demonstrated remarkable proficiency in various coding tasks [16, 31], including code generation [8, 34], software testing [42], program repair [7], and code refinement [12, 15].

Despite their remarkable performance, current LLMs for code generation heavily rely on enormous training data to acquire a broad spectrum of open-domain knowledge. Notably, the evaluation of present LLMs typically revolves around open-domain benchmarks like HumanEval [8] and MBPP [3], which primarily consist of programming contests (e.g., sorting, dynamic programming), and while they showcase the capabilities of LLMs in certain aspects, they do not fully represent the intricacies and challenges associated with real-world code generation scenarios [18, 27].

In this article, we explore a more demanding code generation scenario, focusing on domain-specific code generation. We define *domain-specific code* as source code that is tailored specifically for and can only be applied to a particular domain (e.g., Web and game), typically developed using domain-specific frameworks (e.g., HTTP, RPC, Unreal). Unlike general-purpose code, domain-specific code presents distinct challenges due to the scarce availability of code corpora tailored to a specific domain. This scarcity is not just about the limited amount of training code available; rather, it highlights the relative rarity of domain-specific training data compared to the extensive pre-training data that spans multiple domains. As a result, LLMs may be short of expertise in these specific domains. This scarcity presents a significant obstacle to the task of generating domain-specific code.

We aim to answer the following major questions: (1) How effective are LLMs such as ChatGPT on domain-specific code generation? (2) How to effectively prompt LLMs to produce domain-specific code? In addition, noticed that it is not always straightforward to guide LLMs on domain-specific code repositories, we also explore the following **research question (RQ)**: (3) Can we enhance code generation models for a particular domain? Specifically, how to efficiently integrate domain knowledge into code generation models to enable them to excel in domain-specific code generation tasks?

To answer these questions, we constructed a domain-specific code dataset that involves two distinct domains and two programming languages: Web development in Go and game development in C++. To ensure that the code belonged to the domain we were interested in, we specifically focus on six prominent industry libraries within the chosen domains, namely, Gin, Prometheus, gRPC-go, Unreal Engine, Cocos2d-x, and Bgfx. We curate relevant repositories from GitHub, employing a filtering process that selects code functions utilizing the aforementioned domain libraries.

We first investigate the abilities of several LLMs (ChatGPT [2], PolyCoder [51], and CodeLlama [39]) in specific domains by comparing their performance on general-purpose code corpora and domain-specific code corpora. Our analysis reveals that although LLMs have made remarkable advancements in generating code for open-domain applications, their performance degrades sharply when applied to specific domains. For ChatGPT, the CodeBLEU score drops by 51.48% on average. We particularly notice that this is often caused by a lack of domain knowledge, particularly the misuse of third-party libraries.

We then investigate how to effectively prompt LLMs using domain knowledge. We design several basic knowledge-based prompts and use them to elicit ChatGPT, the most popular LLM for code generation. We experimented with different combinations of the basic prompts to study how different types of prompts influence the performance of domain-specific code generation. We observe result improvement consistently on all library-specific datasets when using knowledge-enhanced prompts such as API sequences and docstrings compared to using the plain function signature prompt.

Based on these findings, we propose a new method called *DomCoder* for domain-specific code generation. *DomCoder* integrates domain knowledge into the code generation process of LLMs through three strategies, including (1) inquiring an external **knowledge GPT (kg-GPT)**, (2) **chain-of-thought prompting (CoT-PT)**, and (3) **chain-of-thought fine-tuning (CoT-FT)**. Kg-GPT trains an external API enquirer based on GPT. Given a function signature, it predicts the API call sequence and uses it to prompt LLM to complete the function body. In the **chain-of-thought (CoT)** strategies, we conceptualize a domain function as a series of sequential states, where each state corresponds to a sub-activity related to an API call to a third-party library. We simulate a CoT process during code generation: At each step, the model generates a "knowledge state" suggesting relevant APIs and a "task state" describing the intended action to be performed. A knowledge state is predicted based on the history of preceding steps, acting as a guide for the code generation at the current step and augmenting domain-specific knowledge. The process continues until the entire programming task is completed. We implement two distinct variants of the CoT strategies, encompassing both the zero-shot and fine-tuning paradigms. Concerning zero-shot code generation (CoT-PT), we sequentially predict APIs using kg-GPT and leverage each API prediction to prompt subsequent states in the sequence. In the fine-tuning setting (CoT-FT), we enrich each of the primary training functions by injecting pre-processed knowledge states. These enriched functions are then used to fine-tune LLMs in an end-to-end fashion.

We apply these strategies on the state-of-the-art code LLMs such as PolyCoder [51], StarCoder [23], and CodeLlama [39] under both zero-shot and fine-tuning paradigms. Our experimental results show that all strategies of *DomCoder* improve the effectiveness of domain-specific code generation under certain settings.

The major contributions of this article are summarized as follows:

- —An empirical study on the ability of LLMs for domain-specific code generation.
- —An investigation of the effectiveness of various prompts for LLMs to generate domain-specific code.
- —A new approach to integrate domain knowledge into LLMs for code generation.
- —A thorough discussion about the implication of our findings and future research directions.

2 Related Work

2.1 Code Generation

Code generation aims to automatically synthesize programs conditioned on programming language context and/or natural language descriptions [4, 8, 51], therefore increasing productivity for developers. Recently, LLMs for code generation have gained significant popularity due to their impressive performance. The state-of-art models are based on GPT, due to their auto-regressive nature, which is suitable for generative tasks where the model predicts the next tokens given the previous context.

General-purpose LLMs exhibit strong code-generation capabilities. These models include GPT-Neo [5], GPT-J [47], and GPT-NeoX [4]. One representative model is OpenAI's ChatGPT [2], a model designed for conversation tasks and supported by a family of backbone models including gpt-3.5-turbo and GPT-4. It is trained on WebText [36], OpenAI's internal corpus collected from Web pages.

There are also many LLMs designed for code-related tasks that are trained on large publicly available code corpora such as the GitHub repositories. For example, Codex [8] is a GPT language model aimed at synthesizing Python programs from docstrings. CodeParrot [44], CodeGen [34], CodeGeeX [56], PolyCoder [51], StarCoder [23], and CodeLlama [39] also belong to this category.

Compared to our works, they mainly focus on general-purpose code generation rather than specific domains.

2.2 Prompting LLMs

With the capability of natural language understanding and generating, LLMs can be prompted to perform a wide range of downstream tasks, such as **question answering (QA)** [9], security [55], and programming [14, 48]. Like fine-tuning, prompting is also a means of transferring pre-trained models to a certain downstream task [19]. When it comes to large models such as GPT-4, prompting is often preferred as it does not require a large amount of training resources.

Prompts can be broadly classified into two categories: soft prompts and hard prompts. Soft prompts are continuous vectors that can be optimized by training. For example, Li and Liang [24] and Lester et al. [20] explored prompt tuning for learning soft prompts. Hard prompts are interpretable text tokens that describe the task and may include exemplars. For example, Reif et al. [37] augmented discrete text prompts to provide more detailed descriptions. Recently, CoT-PT [49] has emerged as a prevalent way to engage LLMs on specific tasks. CoT-PT has leveraged exemplars with intermediate steps to enhance the reasoning capabilities of LLMs in complex reasoning tasks.

The use of prompting techniques has also been extended to code generation [10, 11, 25, 48]. Some researchers optimize the prompting patterns. Liu et al. [25] and White et al. [50] investigated various prompt templates for ChatGPT. Some works enhance prompting by incorporating contextual information. For example, Liu et al. [25] added to the prompt optimization template extra context such as Java Class information. Li et al. [22] introduced test cases and API information to retrieval-based prompts.

While previous studies have used a function as the unit of prompting, our study focuses on more fine-grained knowledge integration strategies by injecting CoT prompts into sub-steps during the code generation process.

2.3 Code Generation with Domain Libraries

Recently, code generation utilizing domain libraries has attracted research attention, and there have been numerous works focused on incorporating knowledge into code generation models [28, 53]. For example, Zan et al. [53] propose a retrieval-then-coding framework where an API retriever first identifies useful APIs and an API Coder generates code using these APIs. Shrivastava et al. [41] introduce domain-specific knowledge in the prompt design process. They present the Repo-Level Prompt Generator framework, which generates example-specific prompts for LLMs. This framework incorporates context from entire repositories. Liu et al. [28] introduce CodeGen4Libs, a technique specifically designed for library-oriented code generation. CodeGen4Libs first generates import statements and then concrete code. Zan et al. [54] leverage the fact that library-oriented code snippets often share similar code sketches. They propose a strategy involving a sketcher and a generator trained on unlabeled data. The sketcher generates library-oriented code sketches, and then the generator predicts code snippets based on them.

Our work differentiates itself from existing studies by exploring deeper integration of domain (API) knowledge into the code-generation process. For example, we propose a GPT-based API recommender that generates API knowledge prompts for LLMs. We also formulate code generation as a CoT process and integrate APIs in each internal coding state. Besides, we provide an in-depth analysis of the knowledge gap of LLMs in generating domain code.

2.4 Studies on Code Generation with LLMs

In addition to our research, numerous empirical studies have explored code generation with LLMs [26, 29, 30, 45]. For instance, Vaithilingam et al. [45] conducted a user study to understand how

Table 1. Statistics of Domain-Specific Libraries

Library	Language	Domain	Description	Popularity	# Train functions
Gin	Go	Web development	Web framework featuring Martini-like APIs	69.5k stars	21,161
gRPC-go	Go	Web development	Remote procedure call across data centers	18.3k stars	129,803
Prometheus	Go	Web development	Client library for application instrumentation	9.1k stars	11,708
Unreal Engine	C++	Game development	Real-time 3D game engine	-	184,808
Cocos2d-x	C++	Game development	Cross-platform 2D game framework	17.2k stars	27,119
Bgfx	C++	Game development	Cross-platform rendering library	13.1k stars	12,285

programmers use and perceive LLM code generation tools. They found that while Copilot reduced the need for online searches, it introduced new challenges in code understanding, editing, and debugging. Mastropaolo et al. [30] evaluated the robustness of existing code-generation LLMs, discovering that developers using different wordings to describe the same code received varying recommendations. Liu et al. [26] hypothesized that existing benchmarks, which rely on curated synthesis problems and test cases, may be insufficient for fully assessing the functional correctness of generated code. They proposed EvalPlus, a code synthesis evaluation framework designed to rigorously benchmark the functional correctness of LLM-generated code. More recently, Liu et al. [29] conducted a systematic empirical assessment of the quality of code generation using ChatGPT, revealing potential issues and limitations in ChatGPT-based code generation.

Our research stands out from previous studies in both its scope and methodologies. Whereas many related studies focus on evaluating the quality and usability of generated code and challenge existing benchmarks, our work emphasizes a crucial aspect of code language models: their proficiency in generating domain-specific code. Specifically, we concentrate on how well LLMs acquire and utilize domain-specific libraries and their overall effectiveness in applying domain knowledge.

3 Experimental Evaluation

In this article, we conduct an empirical study of LLMs on domain-specific code generation. Specifically, we aim to address the following RQs:

RQ1: How effective are LLMs in domain-specific code generation?
RQ2: What are effective methods for prompting LLMs to produce domain-specific code?

3.1 Data Collection

To evaluate LLMs on domain-specific code generation, we collect data from the public repositories on GitHub that involve certain domains. As the domain is not usually explicitly labeled in GitHub repositories, we track the domain of each function through the third-party libraries they utilize. We consider two popular domains in our experiments, including Web and game development. The third-party libraries of these two domains are clear for identification. For each domain, we investigate three libraries provided by a giant Internet company, voted as the most commonly used in their domain applications. Table 1 shows the details of all libraries, including the languages, domains, descriptions, and popularity represented by GitHub Stargazer counts. For Web development, we chose three libraries in the Go language, including Gin, gRPC-go, and Prometheus. *Gin* (`gin-gonic/gin`) is a Web framework that features a Martini-like API with better performance. *gRPC-go* (`google.golang.org/grpc`) is an implementation of gRPC, a high-performance, general-purpose, open source RPC framework designed by Google. *Prometheus* (`prometheus/client_golang`) is a client library for instrumenting Go applications. It has two separate parts, one for instrumenting application code and one for creating clients that talk to the Prometheus HTTP API.

For game development, we chose three C++ libraries: `Unreal Engine`, `Cocos2d-x`, and `Bgfx`. *Unreal Engine* is a real-time 3D game development engine widely used in the industry. With solid network support, it can be used to create multi-player online games that require high performance and low latency. *Cocos2d-x* is an open source multi-platform C++ framework for building graphical applications, especially 2D games. It works on platforms including iOS, Android, macOS, Windows, and Linux. *Bgfx* is an open source cross-platform rendering library that can be used with graphic libraries to build game frameworks.

The detailed data collection process is as follows. We first filter all publicly available repositories with above 50 Stargazer counts by the target language (e.g., `Golang`) using the GitHub repository search API.[1] Then, for each repository filtered, we use the GitHub code search API to check if it contains the library name which must be imported, and if so, retrieve all program files that contain the library name. We extracted and de-duplicated all functions from the collected files, then filtered out those with an empty function body. We also leave out functions with simple names such as *main* or *init*, as they do not contain sufficient context information about the functionality of the code. The statistics of our final dataset are shown in Table 1.

3.2 Evaluated Language Models

We investigate three LLMs: ChatGPT [2], CodeLlama [39], and PolyCoder [51]. They are all widely used for code generation.

ChatGPT [2] is a popular LLM that performs generation tasks in the dialogue format. The model also has demonstrated superb ability in coding. ChatGPT involves a family of backbone models. We chose the most widely used version `gpt-3.5-turbo` to experiment with. The model has around 154B parameters. Since it is not open sourced, we connect to the model through the official API provided by OpenAI[2] once at a time, then retrieve the code segment from its response text.

PolyCoder [51] is a well-established open source model based on GPT-2 and trained on GitHub code. The model demonstrates competitive performance to GPT-3 counterparts. We use the released pre-trained checkpoint PolyCoder-7B.[3]

CodeLlama [39] is the state-of-the-art language model for code generation. CodeLlama is built upon Llama 2, by further training on code-specific datasets. The model involves four sizes with 7B, 13B, 34B, and 70B parameters, respectively. We use the 7B model, which can be served on a single GPU while showing efficient and accurate performance on code completion.

For each model, we set the maximum generation length to 256.

3.3 Evaluation Metrics

We did not adopt the widely used *pass@k* (the passing rate of generated code on test cases) [8] for evaluation. The rationale behind this choice is rooted in the distinctive nature of the domain-specific functions we investigate. Unlike programming contests in existing benchmarks [8, 34], domain-specific functions tend to be notably intricate, often interrelated with multiple functions, designed for specific platforms, or reliant on a lot of third-party libraries. It is too rigorous to have all of them executable, even with powerful LLMs. For example, our preliminary experiments indicate that even ChatGPT achieves a 0% pass rate out of 200 samples when attempting to pass their test cases. Furthermore, testing them necessitates the labor-intensive task of manually crafting hundreds of test cases, thoroughly constructing multiple functions, and configuring reliant libraries

[1] https://docs.github.com/en/rest/search?apiVersion=2022-11-28
[2] ChatGPT: https://platform.openai.com/docs/api-reference
[3] PolyCoder: https://huggingface.co/NinedayWang/PolyCoder-2.7B

Table 2. The Code Generation Performance of LLMs on Open-Domain and Domain-Specific Datasets

Dataset	General-purpose		Code-oriented			
	ChatGPT-3.5-154B		*CodeLlama-7B*		*PolyCoder-2.7B*	
	BLEU	**CodeBLEU**	**BLEU**	**CodeBLEU**	**BLEU**	**CodeBLEU**
Open-domain datasets						
HumanEval	39.00	30.05	18.13	17.99	22.22	13.81
CodeSearchNet (Golang)	11.16	25.29	16.14	19.63	15.97	19.56
Domain-specific datasets						
Gin	5.81	17.73	8.47	18.07	8.13	17.05
Prometheus	1.75	12.17	1.79	13.13	1.77	12.11
gRPC-go	2.53	12.15	57.61	60.39	55.36	57.52
Unreal Engine	5.22	10.57	0.73	10.21	0.94	9.87
Cocos2d-x	3.71	12.92	16.76	25.87	13.83	23.83
Bgfx	0.86	8.09	0.55	7.72	0.76	7.16

and platforms, which is an impractical endeavor. Given these inherent complexities and limitations, conducting evaluations based on real executions is infeasible.

Therefore, in our study, we have chosen to utilize other popular metrics such as BLEU and CodeBLEU, which have also been employed by related works on code generation [21, 28]. BLEU [35] measures sequence similarity by calculating their n-gram matches. CodeBLEU [38] is a metric based on BLEU, adjusted for code evaluation, which introduces syntactic and semantic similarities into comparison. We compute both metrics using the scripts provided by the CodeXGLUE paper.[4]

3.4 RQ1: Effectiveness of LLMs in Domain-Specific Code Generation

3.4.1 Methodology. In this RQ, we investigate the capabilities of LLMs in specific domains. We focus on the code completion scenario, wherein an LLM is presented with a function signature as a prompt, and its objective is to complete the function body. Previous work has indicated that a good function signature already provides enough information to illustrate its functionality [13]. Another reason for not including **Natural Language (NL)** requirements in the prompt is that the domain-specific code we collected contains only a small portion (<5%) of code that contains NL requirements.

We sampled 500 functions from each library-specific dataset we collected (Section 3.1) for the completion. We compare the model-generated code against the ground-truth one in the original function and compute the evaluation metrics.

We compare the quality of LLM-generated code in the domain-specific dataset to that in open-domain (general-purpose) datasets and then analyze the performance gap. We use HumanEval and CodeSearchNet (Golang) [17] as the open-domain code corpora. HumanEval is a widely used benchmark for evaluating code generation models [8, 34]. CodeSearchNet is a benchmark for evaluating code comprehension tasks. The benchmark was collected from miscellaneous projects in GitHub and was processed unbiasedly.

3.4.2 Quantitative Results. Table 2 shows the quantitative results. Overall, all three models demonstrate a decline in performance on domain-specific datasets compared to open-domain datasets, which conforms to our hypothesis. For ChatGPT-3.5, the average BLEU score on

[4] https://github.com/microsoft/CodeXGLUE/tree/main/Code-Code/code-to-code-trans/evaluator/CodeBLEU

Reference code:

```
func (api *API) RegisterRouter(router *gin.RouterGroup){
    v1 := router.Group(constants.APIVersion1)
    api.execute.Register(v1)
}
```

ChatGPT:

```
func (api *API) RegisterRouter(router *gin.RouterGroup){
    router.GET("/test", api.Test)
}
```

(a) Misused API calls

Reference code:

```
func Middleware(app newrelic.Application) gin.HandlerFunc{
  return func(c *gin.Context) {
    name := c.HandlerName()
    w := &headerResponseWriter{w: c.Writer}
    txn := app.StartTransaction(name, w, c.Request)
    defer txn.End()
    c.Set("txn", txn)
    c.Next()
  }
}
```

ChatGPT:

```
func Middleware(app newrelic.Application) gin.HandlerFunc{
  return func(c *gin.Context) {
    txn := app.StartTransaction("test", nil, nil)
    defer txn.End()
    c.Set("txn", txn)
    c.Next()
  }
}
```

(b) Missing API calls

Fig. 1. Code examples generated by ChatGPT.

domain-specific datasets drops by 70.35% compared to the score on CodeSearchNet, while the average CodeBLEU drops by 51.48%.

We have noted that non-popular libraries (such as Prometheus) tend to experience a more significant performance decline compared to others. This is likely due to the fact that there is less open source code that utilizes the domain-specific library available for LLM training. On the contrary, gRPC-go and Cocos2d-x exhibit exceptionally high performance in code-oriented LLMs. Upon manual examination, we observe monotonous code patterns in functions that utilize these two libraries, such as gRPC server handler functions and Cocos2d-x interface functions from C++ to Lua. Such repetitive knowledge is more easily learned by the models.

Notably, ChatGPT outperforms the code-oriented LLMs in the open-domain dataset. This phenomenon can be explained by the fact that general-purpose LLMs are trained on a vast amount of diverse data, which enables them to learn a broad range of knowledge. This makes them better suited for open-domain tasks requiring a more general language understanding.

3.4.3 Qualitative Results. To find out which aspects of the models fall short of domain-specific code generation, we also qualitatively examined the ChatGPT-generated code manually. We randomly sample 200 functions from the library-specific dataset. For each function, we use the function signature as a prompt and let ChatGPT complete the function body. We check if the generated code has implemented the desired functionality as in the original reference code, and we pay particular attention to the invoke of domain libraries.

Overall, of the 200 model-generated functions sampled, we observed that 32 functions were correct (16%), 37 implemented functionality completely different from the reference code (18.50%), 52 had problems with API misuse (26%), and 79 had problems with missing APIs (39.50%). We hereby summarize common mistakes that appear in the model-generated code (Figure 1).

(1) *Misused API Calls.* The generated code is similar in format to the reference code, but the actual functionality is incorrectly implemented due to wrong API calls, we refer to this kind of mistake as *misused API calls*. An example is shown in Figure 1(a). The reference function *RegisterRouter* registers an HTTP API router, with a *gin.RouterGroup* object passed as a parameter. The correct API to be called is *router.Group*, which returns a router group for later use, while the model-generated code used *router.GET*, which is also an API of the Gin library, but conducts a different activity: handling a GET request. Therefore, the generated code functionality is inconsistent with the demand given by the function signature.

(2) *Missing API Calls.* In some cases, we observed the model-generated code is missing one or several APIs compared to the reference code, resulting in a deviation of the implemented functionality from the expected functionality. We refer to this issue as missing API Calls. As shown in Figure 1(b), the reference code invokes *gin.Con text.HandlerName* to retrieve a name for the handler and passes it as a parameter to *app.StartTransaction*. While in the model-generated code, this API call is missing, leading to wrong parameters for the next API call.

In summary, most of the mistakes in domain-specific code generation are related to API misusage, leading to our conclusion that existing LLMs are unfamiliar with the API usages of certain domain-specific libraries.

Finding 1: LLMs exhibit sub-optimal performance in generating domain-specific code, specifically due to their limited proficiency in utilizing domain libraries.

3.5 RQ2: Prompting LLMs for Domain-Specific Code Generation

Based on the evaluation results in RQ1, we have found that LLMs underperform in the domain-specific code generation task, which we attribute to LLMs' lack of domain-specific knowledge by observation. Therefore, we assume that incorporating domain-specific knowledge into LLMs can improve the quality of code generation. In the view of programming, domain knowledge typically refers to the usage of third-party libraries or packages, namely, API documentation. For instance, in Web development using Golang, the Gin Web framework is commonly used to enhance productivity and performance. The details of the Gin API, such as the names and descriptions of its functions, are examples of domain-specific knowledge.

Another question is how to incorporate domain knowledge into LLMs. As a direct and cost-effective method, prompting has emerged as one of the most popular ways to engage with LLMs [6, 25, 50]. It provides LLMs with a few instructions such as intentions, demonstration examples, and a CoTs, thereby enabling LLMs to produce desired outcomes. We hypothesize that by formulating domain knowledge as prompts, we can effectively induce LLMs to generate domain-specific code more proficiently.

Therefore, in RQ2, we investigate how to effectively prompt LLMs using domain knowledge and how different forms of prompt impact the performance of domain-specific code generation. To address this, we designed several basic knowledge-based prompts and used them to elicit ChatGPT, the most popular LLM for both text and code generation. ChatGPT also demonstrates a superb ability in understanding human prompts; hence, it is easier to showcase the effects of different prompts. We study the effect of each kind of knowledge prompt separately. Moreover, we experimented with different combinations of the basic prompts to study how the order of knowledge elements can affect the code generation quality and how different types of knowledge can complement each other.

3.5.1 Prompt Design. We regard the function signature as a plain prompt without any knowledge injection. Apart from this, we designed three types of knowledge-based prompts to enhance the code generation quality:

(1) *Library import prompt:* a phrase specifying the third-party library to be used.
(2) *API prompt:* a list of APIs to be called to implement a function, retrieved from the ground-truth code.
(3) *Docstring prompt:* the natural language descriptions of the APIs, retrieved from the library documentation.

Table 3. Examples of Knowledge-Enhanced Prompts

Prompt type	Example
Basic prompts	
Function signature	`Complete this function: func Routes(r *gin.Engine)`
Library import	`Complete this function using gin: func Routes(r *gin.Engine)`
API	`Complete this function using gin.RouterGroup.Use: func Routes(r *gin.Engine)`
Docstring	`gin.RouterGroup.Use adds middleware to the group.` `Complete this function: func Routes(r *gin.Engine)`
Combined prompts	
API + docstring	`Complete this function using gin.RouterGroup.Use.` `gin.RouterGroup.Use adds middleware to the group. func Routes(r *gin.Engine)`
Docstring + API	`gin.RouterGroup.Use adds middleware to the group.` `Complete this function using gin.RouterGroup.Use: func Routes(r *gin.Engine)`

Table 4. Results of Prompting ChatGPT for Domain-Specific Code Generation (CB Denotes CodeBLEU)

	Gin		Prometheus		gRPC-go		Unreal Engine		Cocos2d-x		Bgfx	
Prompt type	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**
Function signature	5.81	17.73	1.75	12.17	2.53	12.15	5.22	10.57	3.71	12.92	0.86	8.09
+ Library import	6.08	17.83	3.37	15.12	5.44	16.93	5.79	11.27	5.37	14.68	1.02	8.79
+ API	**14.86**	26.69	**7.18**	20.51	**22.24**	**31.84**	**14.78**	18.34	**14.18**	**22.35**	**1.39**	**11.32**
+ Docstring	14.56	**27.04**	7.04	**20.78**	22.07	31.74	14.58	**18.35**	14.06	**22.35**	1.33	**11.40**
+ Docstring	5.57	17.78	1.94	12.31	2.54	12.33	5.32	10.41	3.57	12.93	0.88	8.38
+ API	13.64	25.94	6.95	20.48	22.08	31.64	14.12	17.74	14.00	22.24	1.23	11.03

Numbers in bold denote the best results.

We also constructed two combined knowledge-based prompts: *API + docstring* and *docstring + API*. To illustrate, we provide examples of each type of knowledge-enhanced prompt for ChatGPT-based code generation in Table 3.

3.5.2 Results. Table 4 shows the performance of ChatGPT on library-specific datasets under various prompt types. We use plain function signatures as a baseline and experiment with each knowledge-enhanced prompt.

We observe result improvement on library-specific datasets when using knowledge-enhanced prompts such as library indication and API name sequence compared to using the plain function signature. Comparing the results of basic prompts, API sequence demonstrates a significant improvement, whereas library import and docstring exhibit relatively modest enhancements. The API name sequences are closer related to the code content and have a more direct effect on code generation. In contrast, the library import prompts do carry concise and useful information, but in meager quantity. The API docstrings contain implicit knowledge in natural language, different from the target programming language, which could be indirect. Furthermore, we observed that the presence of docstrings alongside APIs does not consistently enhance performance when compared to using APIs alone, despite occasional cases where such combined prompts achieve the highest scores. This discrepancy may be attributed to the long text of the docstrings, which sometimes hinders the comprehension of APIs when incorporated into the prompt.

For two combined prompts that have the same knowledge elements but in a different order, we can see they have similar results for code generation enhancement, but API sequence + docstring prompt is slightly better, indicating that the orders in which knowledge is organized do affect the results.

Finding 2: The performance of domain-specific code generation can be improved by prompting LLMs with domain knowledge, particularly the usage of domain libraries.

4 Code Generation with Domain Knowledge Integration

In the previous RQs, we have demonstrated that injecting API knowledge into prompts has a positive effect on code generation. However, in real-world application scenarios, the model needs to perform the prediction based on the input code context alone, which does not contain any prior API knowledge. This sparks an idea of automatically incorporating knowledge into code generation: We can extend code LLMs by constantly inquiring about API knowledge from a knowledge base and using it to prompt the next code fragments. In this section, we experiment with three strategies to integrate knowledge into code-generation LLMs and compare their effectiveness. Specifically, we aim to address the following RQs:

RQ3: How to effectively integrate domain knowledge in the code generation process?

- *RQ3.1:* Is API recommendation useful for automatically prompting LLMs to generate domain-specific code?
- *RQ3.2:* Is CoT useful for automatically prompting LLMs to generate domain-specific code?
- *RQ3.3:* By fine-tuning, can we further enhance the effectiveness of CoT for LLMs?

We introduce the three experimental strategies in the following sections and then present our empirical results.

4.1 Prompting LLM by Inquiring External Knowledge

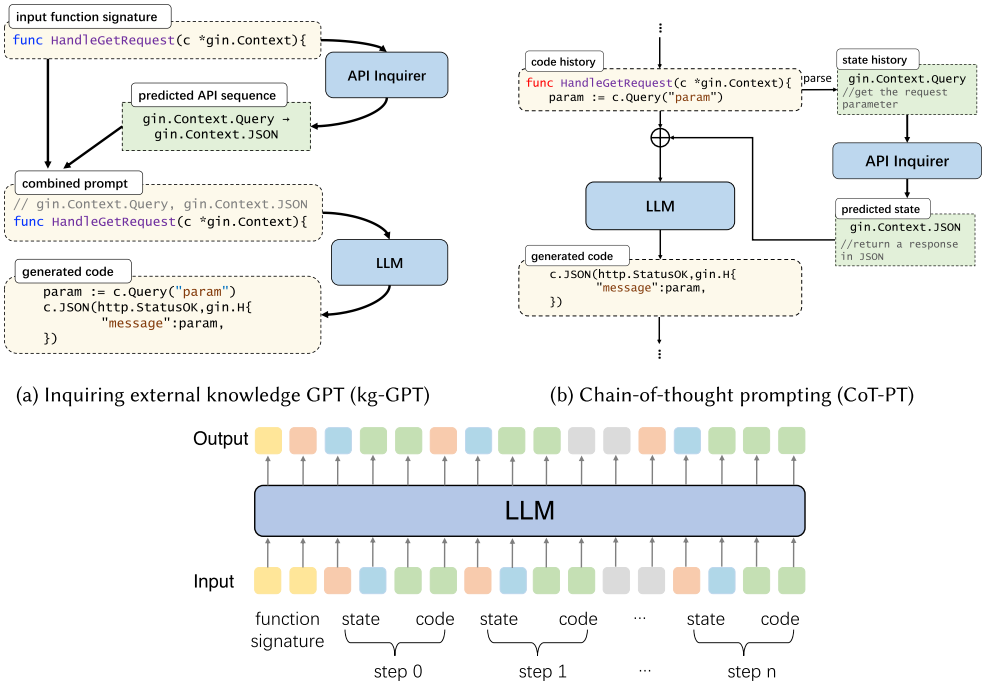
Perhaps the most straightforward approach for injecting knowledge into a code LLM is to train an external knowledge inquiring model named kg-GPT. This strategy consists of two modules, one for API knowledge inquiry and the other for code generation, as illustrated in Figure 2(a). Given the input function signature, the API knowledge inquirer recommends a sequence of possible APIs that could be used to implement a function. Then, the LLM takes the predicted API sequence and the input function signature as a prompt to generate the function body.

The API inquirer is realized as a GPT trained using causal language modeling, and each training sequence is composed of a function signature and its corresponding API usages. In this way, it can generate a list of APIs by predicting the next sequence when given the function signature as a context. To introduce the domain-specific API knowledge, we add the predicted API list as a line of comment above the function signature, forming a combined prompt. We then feed the hint to the LLM, which generates the function body. Owing to the API knowledge in the prompts, it inclines to generate code containing the recommended APIs.

4.2 Prompting LLMs with CoT

While kg-GPT accomplishes the initial goal of incorporating domain knowledge into LLMs, it does so in a shallow fashion. We hypothesize that a more fine-grained integration of domain knowledge into the generation process has the potential to improve the overall generation performance. Inspired by the CoT-PT [49] for LLMs, we experiment with the second strategy that incorporates knowledge into the code generation process in a CoT manner.

4.2.1 Programming as a CoT Process. To deal with a complex task, one often adopts a multi-step thinking strategy, which is to break down the original task into a series of sub-tasks and solve them step by step. This thinking process is known as *CoT* [49]. CoT-PT has been proven to be effective in improving the reasoning capabilities of LLMs [19, 40].



(a) Inquiring external knowledge GPT (kg-GPT)

(b) Chain-of-thought prompting (CoT-PT)

(c) Chain-of-thought fine-tuning (CoT-FT)

Fig. 2. Illustration of the three experimented strategies for knowledge integration.

Like reasoning, programming is also a task that requires careful logical thinking [52]. When faced with a programming task to implement a complex requirement, one often thinks in a CoT manner [10]. This is natural since a complete code segment (e.g., a function) can usually be decomposed into multiple sub-activities [43], each containing one or a few statements. We refer to a sub-activity as a *step* in the coding process. At each step, one decides what action should be taken based on former contexts and how it should be implemented. For domain-specific code, this decomposition of activities is much related to API calls of third-party libraries [32, 33]. An API acts like an intermediary, providing developers with functionality implemented by a third-party library. One statement calling an API can be regarded as a separable sub-activity.

Having formulated code generation as a chain of thinking steps, we attempt to integrate knowledge into each step through prompting. We introduce *knowledge states* to simulate the implicit states in the thinking process. A knowledge state corresponds to a short code sub-segment, it consists of an API state and a task state. The API state suggests one or more APIs to be used to accomplish this sub-task, and the task states describe the action to be performed. Combined together, they summarize the chain of operations bound to each step.

The generation process involves three sub-tasks: understanding current input, including formerly written code and corresponding states, predicting the next knowledge state, and generating the next code snippet. The three steps are executed in a loop until we have generated a complete function body or the maximum generation length is reached.

Figure 3 shows an example of a function generation process in a CoT manner. We take a function signature as input, where the function name `HandleGetRequest` indicates that the requirement is to handle GET requests, and the parameter `c *gin.context` points to the *Gin* library. Based on the input, the model generates code step by step. At step 0, it first predicts the API

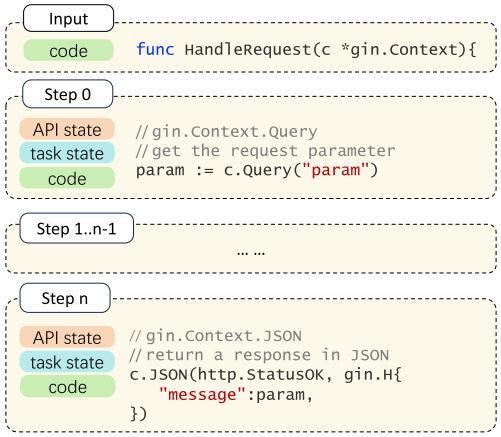


Fig. 3. A CoT view of code generation.

state `gin.Context.Query` and the task state *get the request parameter*. The states suggest that the following code should perform the task of retrieving the request parameter using the API `gin.Context.Query`. Then conditioned on the code history (which is the function signature for step 0) and the current state, the model generates code for step 0, `param := c.Query("param")`. Similarly, the model continues to recursively predict code with the states at each step, until the end of the function is reached. If no API is required, we mark it as an empty state and simply skip the API prediction for this step.

4.2.2 Overall Design. Figure 2(b) illustrates the design of this strategy. It consists of two modules, one for prompt generation and the other for code generation. The prompt generator is realized with an external knowledge enquiring model kg-GPT as shown in Figure 2(a), it predicts the next possible API based on the API state history or input function signature. For each step, the prompt generator predicts an API state based on the state history. The corresponding API docstring is appended as the task state. The generated API and task states are combined with the code history and used as a prompt for the LLM, which then predicts code for the current step.

4.3 CoT-FT

When prompting with CoT, the prompt generation module relies only on API state history for prediction, which lacks information on code syntax and structure, affecting the prediction accuracy. To improve the quality of predicted APIs, we experiment with the third strategy to predict API state conditioning on code history, namely, CoT-FT, as illustrated in Figure 2(c). In this strategy, we combine all sub-steps together and fine-tune a single language model in an end-to-end fashion. For each function, we construct a training sequence, which consists of a user input function signature and an output code segment decomposed as n steps. At a step t, the model reads the history H_{t-1}, then generates a knowledge state K_t and a code snippet C_t, concatenated as $S_t = [K_t, C_t]$. The history includes the user input I_0 and all previously generated $t-1$ steps $[S_0, S_1, ...S_{t-1}]$:

$$H_{t-1} = [I_0, S_0, S_1, ...S_{t-1}].$$

Conditioned on the history, the model first predicts a knowledge state K_t, which consists of an API sequence and a task description:

$$K_t = LM(H_{t-1}).$$

Table 5. Examples of API Knowledge

API name	API docstring
gin.BasicAuth	Returns a Basic HTTP authorization middleware
gin.Context.Abort	Prevents pending handlers from being called
gin.Context.JSON	Serializes a given struct as JSON

This knowledge state is then appended to the history, forming a complete context. The model conditions on this input to generate code for step t:

$$C_t = LM([H_{t-1}, K_t]).$$

Combining each step, the overall code generation process is shaped as

$$K_0, C_0, K_1, C_1, ...K_n, C_n = LM(I_0).$$

After removing the states, the final generated code is obtained:

$$C = C_0, C_1...C_n.$$

We adopt the end-to-end training method by combining all steps as a single training sequence and fine-tuning a causal language model (i.e., Transformer decoder) [46]. A complete training sequence is concatenated by one user input and all sub-sequences, shaped like $[I_0,\ S_0,\ S_1\ ...\ S_n]$. By causal language modeling, the model is trained to predict the probability of the next token based on all former tokens. In this way, we can model the joint probability over the entire sequence.

4.4 Experimental Setup

Collecting API Knowledge. To evaluate code generation with domain knowledge, we extend the domain-specific dataset for RQ1 and RQ2 (Section 3.1) with domain knowledge. We are concerned with knowledge about certain libraries, including the APIs and their corresponding natural language docstring, organized in a dictionary format. As Table 5 depicts, an API name is a unique identifier of a specific function in a third-party library, and an API docstring provides a supplementary and more detailed explanation of its behavior.

We obtain API calls from the collected functions in the original dataset. For each function, we parse the code using *tree-sitter*.[5] We identify all <variable name, variable type> pairs in a function from the parameter definition and variable declaration statements; if the variable type is a struct or a class defined in the target library, we mark the corresponding variable as targeted. We then identify all call expression statements and extract the names of the called function. If the call expression contains our target package/class identifier or the called function is initiated by a target variable, we mark it as a target API. API names are then used as queries to retrieve document strings from the knowledge base.

We obtain API docstring either from the API documentation or from the library source code. For libraries with comprehensive official API reference documentation, including `Unreal Engine`, `Cocos2d-x`, and `Bgfx`, we extract knowledge directly from the document by crawling the Web page, then keep the pairwise API declarations and descriptions. For libraries without a document, including `Gin`, `gRPC-go`, and `Prometheus`, we use *tree-sitter* to parse the source code of the library, extracting each function with a corresponding docstring if there exists one. We use the function name as the API name and docstring as the API description.

[5] https://tree-sitter.github.io/tree-sitter/

The parsed API calls and docstrings are integrated into each function for the CoT-FT. To implement knowledge integration while conforming to the programming language format, we add the API call and docstrings as comments above each corresponding call expression statement.

Backbone Models. We employ PolyCoder [51], StarCoder [23], and CodeLlama [39] as backbone LLMs, which are well-established open source models trained on GitHub code. Considering our computational resources, all fine-tuning experiments were only carried out on PolyCoder. PolyCoder is based on GPT-2, which is easy to develop and deploy. More importantly, it demonstrates competitive performance to GPT-3 counterparts [51]. We employ the released checkpoint of pre-trained PolyCoder-2.7B, StarCoder-15.5B,[6] and CodeLlama-7B[7] for all experiments. We implement the API inquirer (kg-GPT) by employing a standard GPT-2 model with 12 layers and 768 dimensionality.

Training and Prediction. We train all models in a server with 8 Nvidia A100 (40 GB) GPU cards. To fine-tune the backbone PolyCoder, we follow the hyper-parameter configurations in the original paper [51], with a batch size of 32 and a learning rate of 1.6e-4. We train the API inquirer with a batch size of 16 and a learning rate of 5e-5.

We randomly sample 500 functions from each dataset to conduct the prediction and evaluation. For each function, we configure the model to generate new tokens until we either encounter the termination token <EOF> or reach a maximum token length of 256, then truncate the output to the end of the function. For the CoT models, we do not count the tokens occupied by knowledge states in the overall length.

Comparison Methods. We compare our approach with all backbone LLMs in the zero-shot setting, which refers to prompting the pre-trained checkpoint without any further fine-tuning. We also compare *DomCoder* with a few-shot learning method proposed by Ahmed and Devanbu [1], which prepend a few in-context examples in the LLM context. We randomly sample 10 in-context examples that invoke the same library to the target domain. Besides the zero-shot setting, we are also concerned with the fine-tuning strategy of our method. Specifically, we compare *DomCoder* (CoT-FT) with PolyCoder fine-tuned on the domain-specific dataset.

4.5 Results

The evaluation results for all methods and strategies are shown in Table 6. Under both the zero-shot and the fine-tuning settings, BLEU and CodeBLEU scores rise when the model is knowledge-enhanced, and the CoT-FT strategy reaches the best scores.

Under the zero-shot setting, both the kg-GPT and CoT-PT strategies improve the results, with kg-GPT demonstrating better performance. API knowledge introduced by kg-GPT raised the BLEU scores by 49.33% on average and the CodeBLEU scores by 9.82%. Under the fine-tuning setting, CoT raised the BLEU scores by 17.10% on average and the CodeBLEU scores by 4.20%. Furthermore, we can see that the performance is improved more significantly on datasets with poor zero-shot results, such as `prometheus` and `Unreal Engine`. Comparatively, the CoT-FT strategy provides modest improvement, except for the CodeBLEU scores for some libraries such as `gin` and `prometheus`. Both kg-GPT and CoT-PT incorporate API knowledge into the zero-shot PolyCoder, and the differences in results could be attributed to their distinct knowledge-integration strategies. Kg-GPT predicts all states of APIs at once, whereas CoT-PT predicts APIs incrementally based on historical states, which do not contain information about code syntax and structure, resulting in unstable quality of the predicted APIs.

Domain fine-tuning improves the results compared to zero-shot, as it allows the model to learn about domain-specific code most straightforwardly. Still, by injecting the knowledge into training

[6] https://huggingface.co/bigcode/starcoder
[7] https://huggingface.co/codellama/CodeLlama-7b-hf

Table 6. Performance of Various API Knowledge Integration Strategies (CB Denotes CodeBLEU)

Model	Gin		Prometheus		gRPC-go		Unreal Engine		Cocos2d-x		Bgfx	
	BLEU	**CB**	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**	**BLEU**	**CB**
PolyCoder (zero-shot)	8.13	17.05	1.77	12.11	55.36	57.52	0.94	9.87	13.83	23.83	0.76	7.16
+In-context learning [1]	8.18	16.36	2.01	12.57	55.54	57.11	0.96	0.98	13.27	23.00	0.83	7.24
+ *DomCoder* (kg-GPT)	**9.91**	18.16	**2.29**	12.76	**56.87**	**58.30**	1.26	**10.20**	**15.33**	**24.09**	**0.78**	7.34
+*DomCoder* (CoT-PT)	9.17	17.41	2.21	**12.65**	55.97	57.70	**1.38**	10.15	16.13	24.91	**0.91**	**7.87**
PolyCoder (fine-tuning)	20.62	27.18	5.88	17.11	62.97	63.37	1.57	11.10	27.65	32.67	1.01	11.80
+*DomCoder* (CoT-FT)	**21.12**	**27.51**	**7.59**	**19.13**	**63.23**	**63.83**	**2.26**	**11.69**	**29.30**	**33.99**	**1.05**	**11.95**
StarCoder (zero-shot)	8.46	17.55	1.68	12.37	58.69	60.09	0.58	9.00	14.08	23.56	0.80	7.21
+In-context learning [1]	7.11	17.32	2.00	12.75	58.12	59.07	1.14	10.76	18.62	26.55	1.12	7.93
+*DomCoder* (kg-GPT)	**13.12**	**20.44**	**3.75**	14.74	**60.41**	**61.85**	**1.74**	**10.98**	**22.43**	**27.53**	**1.20**	**8.67**
+*DomCoder* (CoT-PT)	12.46	19.77	3.32	13.66	59.91	61.25	1.64	10.96	20.78	25.81	1.17	8.42
CodeLlama (zero-shot)	8.47	18.07	1.79	13.13	57.61	60.39	0.73	10.21	16.76	25.87	0.55	7.72
+In-context learning [1]	8.22	18.58	2.69	13.87	**59.35**	**61.49**	0.81	10.35	17.90	27.06	0.60	8.47
+*DomCoder* (kg-GPT)	11.71	19.83	2.62	13.41	55.93	58.24	0.99	10.55	**19.58**	**27.71**	**0.65**	**8.66**
+*DomCoder* (CoT-PT)	**11.75**	**19.90**	**3.07**	**14.58**	56.03	58.69	**1.15**	**11.15**	16.39	25.16	0.57	8.21

Numbers in bold denote the best results.

Table 7. Performance of Kg-GPT in API Recommendation

Model	Gin		Prometheus		gRPC-go		Unreal Engine		Cocos2d-x		Bgfx	
	BLEU	**Hitratio**	**BLEU**	**Hitratio**	**BLEU**	**Hitratio**	**BLEU**	**Hitratio**	**BLEU**	**Hitratio**	**BLEU**	**Hitratio**
PolyCoder	16.81	0.03	8.02	0.03	32.09	0.37	7.47	0.04	30.10	0.33	3.32	0.01
CodeLlama	21.64	0.06	12.42	0.04	36.92	0.38	9.50	0.07	35.91	0.36	6.73	0.03
Kg-GPT	**46.48**	**0.36**	**27.16**	**0.20**	**52.67**	**0.67**	**16.48**	**0.09**	**57.24**	**0.50**	**34.25**	**0.28**

Numbers in bold denote the best results.

data in a CoT fashion, the performance is raised to a higher level. We note that the improvement that CoT-FT brings to PolyCoder is relatively marginal. Nevertheless, it demonstrates that the proposed CoT knowledge integration is also useful in the fine-tuning mode despite still having ample room for further improvement.

Note that, in most cases, BLEU scores are smaller than CodeBLEU scores. Particularly, in Unreal Engine and Bgfx, CodeBLEU score is 10 times of BLEU score. It is caused by the metric design: BLEU only evaluates the n-gram match, while CodeBLEU examines a combination of the n-gram match, syntax match, and dataflow match. In practice, a low n-gram match could come with a high syntax match and dataflow match, especially when the generated code has correct functionality but different variable names.

As an essential part, we assess the accuracy of the APIs recommended by kg-GPT. This evaluation holds significance since it influences the language model's performance in code generation relying on the suggested API sequences. We employ the same dataset outlined in Section 4.4 for training the kg-GPT. We measure the quality of API recommendations using BLEU and Hit Ratio. The Hit Ratio refers to the percentage of correctly recommended APIs out of the total number of predicted APIs. A higher Hit Ratio indicates a better performance of the API recommendation by the kg-GPT. For comparison, we provide the results of API recommendation by PolyCoder-2.7B. These results are obtained by extracting API sequences from the predicted code by PolyCoder. The results are presented in Table 7. We obverse that kg-GPT recommends domain API sequences with high accuracy. For example, it achieves 10 times the accuracy in the `Bgfx` library in terms of BLEU score.

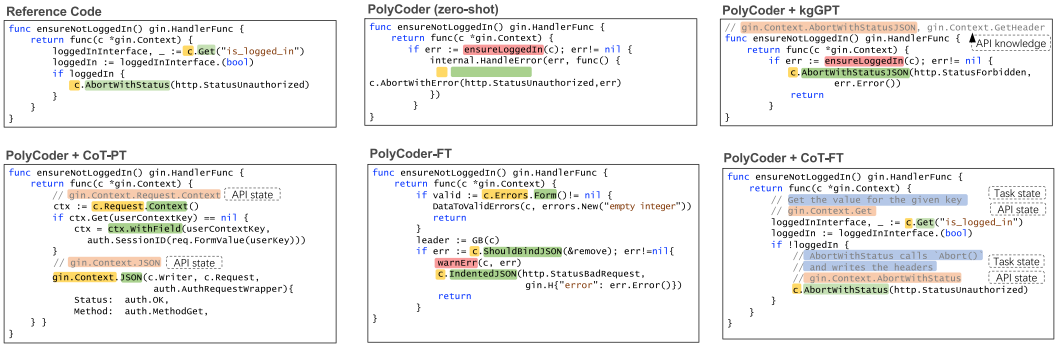


Fig. 4. Examples of generated code.

4.6 Qualitative Analysis

We inspected the code generated by various methods and presented cases to illustrate their effectiveness. Figure 4 presents an example of code generation by various methods.

We take a function using the Gin library as an example. As its name indicates, the reference function ensures the logged-in status is False. It checks the logged-in status using `gin.Context.Get` and aborts using `gin.AbortWithStatus` if it's True.

For this function, we compare the results of all strategies. Among them, the CoT-FT strategy generates the best result. Through step-by-step generation, it first predicts the task description and API name as the knowledge state and then generates code to implement the task in the description using the API. Since it correctly predicts the state at each step, it eventually generates the correct code segment that meets the requirements. Comparatively, all other methods give more or less erroneous results. For example, the code generated by the PolyCoder-FT called several APIs of the targeted gin library, but none of them is correct, failing to fulfill the requirements.

Finding 3: The performance of domain-specific code generation can be improved through domain knowledge integration in the generation process of LLMs.

4.7 Industry Study

To fully assess the effectiveness of *DomCoder*, we perform an in-house industry study on Tencent Inc, a world-famous IT company. Our study focuses on the domain of messenger Apps (e.g., WeChat, QQ) which are the predominant products of this company. We start by creating a survey about the domain libraries that are mostly adopted by the developers in the group of messenger products. Results show that an internal library called tRPC-go (https://github.com/trpc/trpc) ranked as the most popular one that is utilized by the group. We therefore selected functions that utilize this library and evaluate the performance of *DomCoder* in completing these functions. To ensure that the selected functions are representative and distinct, we specifically chose functions containing three to seven APIs. Functions with identical API sequences were clustered into groups. These function groups were then sorted based on the number of functions they contained, in descending order. From each class, we randomly selected one function. Additionally, we manually filtered out functions that developers may find challenging to understand. As a result, we obtained a set of 30 test functions for evaluation.

We built *DomCoder* based on CodeLlama-7B and applied it to complete the selected test functions. We test the kg-GPT strategy which demonstrates the best performance in the quantitative study and is easier to deploy. We measure the performance using two metrics: (1) *task relevance*, which

Table 8. Performance of *DomCoder* in the Industry Study

Model	Task relevance	Semantic correctness
CodeLlama-7B (zero-shot)	20/30	6/30
- w/ *DomCoder* (kg-GPT)	28/30	18/30

means that the generated content satisfies the intention in the function signature, and (2) *semantic correctness*, which means that the implemented function body is semantically correct. We asked three developers from the relevant team in Tencent to rate the evaluation criteria independently. Conflicts were resolved through discussion until a consensus was reached.

The results are presented in Table 8. We find that *DomCoder* significantly enhances the quality of generated code. The semantic correctness of the generated code by *DomCoder* is triple that of the code generated by the original CodeLlama-7B. The results confirm the effectiveness of *DomCoder* in wider application scenarios and metrics in the industry.

5 Discussions

5.1 Future Directions

Based on our findings, we suggest three possible future directions for domain-specific code generation by LLMs.

First, programming knowledge can extend beyond API documentation. We suggest future research to consider more sources of domain knowledge in the design of prompts and the integration of knowledge. Regarding syntax, more comprehensive information about third-party libraries could be helpful, including class descriptions, relationships between classes, and other relevant information. For code semantics, descriptive and explanatory information from sources like Stack Overflow could be useful. Moreover, knowledge about a certain domain such as workflows and code templates could also be useful.

Second, results on RQ2 demonstrate that docstrings also play an important role in the design of prompts. But this presents a new challenge of increasing the code length. In addition, the ability to understand and generate natural language descriptions is relatively limited for code-oriented models. We suggest future research to design better mechanisms to integrate knowledge into code, bridging the information gap between programming language and natural language. For example, we can consider pre-training tasks that jointly predict code and knowledge.

The results of the kg-GPT strategy suggest that a separate knowledge enquirer can be an effective way to incorporate knowledge with code. In the future, researchers can focus on designing more sophisticated knowledge inquirers. For example, they can build a QA or search engine that detects any factual knowledge and provides the prompt during the code generation process.

The results also show that CoT is an efficient way to incorporate knowledge into LLMs in both fine-tuning and usage stages. In the future, researchers could explore integrating knowledge in the early stage of LLM training. Instead of training a casual language model on plain code, they could augment code with API knowledge and use it in pre-training. They could also decompose source code into sub-modules, each representing a specific sub-activity, and generate the modules in a certain order as a simulation of CoT.

5.2 Threats to Validity

The empirical study in this research investigates three LLMs. In particular, the code-specific LLMs we studied are limited to a parameter size not exceeding 2.7B. Considering the rapid development

of LLMs, the LLMs studied in this article may not fully represent all the latest advancements in large-scale models. Furthermore, while our investigation primarily focuses on code generation tasks in the domains of Web and game development, it is essential to recognize that code generation is a ubiquitous task across a wide range of domains. Therefore, the findings obtained from the two particular domains can potentially offer general insights applicable in various contexts. Regarding the evaluation metrics, we have utilized BLEU and CodeBLEU. Although these two metrics are widely used in code intelligence research, they may not fully represent human experience.

6 Conclusion

In this article, we conduct an empirical study on domain-specific code generation by LLMs. Our study finds that LLMs such as ChatGPT exhibit sub-optimal performance in generating domain-specific code, and adding knowledge prompts about domain libraries improves the performance. To further investigate how to incorporate API knowledge into LLMs, we design three experimental strategies to automatically prompt LLMs, including inquiring about an external kg-GPT, CoT-PT, and CoT-FT. Our experimental results show the effectiveness of knowledge integration in both zero-shot and fine-tuning settings. We hope our study can inspire future work on improving the code generation capabilities of LLMs for specific domains.

References

[1] Toufique Ahmed and Premkumar Devanbu. 2022. Few-shot training LLMs for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 1–5.

[2] Open AI. 2023. ChatGPT. Retrieved from https://openai.com/blog/chatgpt

[3] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, et al. 2021. Program synthesis with large language models. arXiv:2108.07732. Retrieved from https://arxiv.org/abs/2108.07732

[4] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, et al. 2022. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of BigScience Episode# 5–Workshop on Challenges & Perspectives in Creating Large Language Models*, 95–136.

[5] Sid Black, Leo Gao, Phil Wang, Connor Leahy, and Stella Biderman. 2021. GPT-Neo: Large scale autoregressive language modeling with mesh-tensorflow. DOI: https://doi.org/10.5281/zenodo.5297715

[6] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D. Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. 2020. Language models are few-shot learners. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 33, 1877–1901.

[7] Jialun Cao, Meiziniu Li, Ming Wen, and Shing-chi Cheung. 2023. A study on prompt design, advantages and limitations of ChatGPT for deep learning program repair. arXiv:2304.08191. Retrieved from https://arxiv.org/abs/2304.08191

[8] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from https://arxiv.org/abs/2107.03374

[9] Xiusi Chen, Yu Zhang, Jinliang Deng, Jyun-Yu Jiang, and Wei Wang. 2023. Gotta: Generative few-shot question answering by prompt-based cloze data augmentation. In *Proceedings of the 2023 SIAM International Conference on Data Mining (SDM '23)*. SIAM, 909–917.

[10] Yu Cheng, Jieshan Chen, Qing Huang, Zhenchang Xing, Xiwei Xu, and Qinghua Lu. 2024. Prompt sapper: A LLM-empowered production tool for building AI chains. *ACM Transactions on Software Engineering and Methodology* 33, 5 (June 2024), Article 124, 24 pages. DOI: https://doi.org/10.1145/3638247

[11] YunSeok Choi and Jee-Hyong Lee. 2023. CodePrompt: Task-agnostic prefix tuning for program and language generation. In *Findings of the Association for Computational Linguistics (ACL '23)*, 5282–5297.

[12] Kayla DePalma, Izabel Miminoshvili, Chiara Henselder, Kate Moss, and Eman Abdullah AlOmar. 2024. Exploring ChatGPT's code refactoring capabilities: An empirical study. *Expert Systems with Applications* 249 (2024), 123602.

[13] Xi Ding, Rui Peng, Xiangping Chen, Yuan Huang, Jing Bian, and Zibin Zheng. 2024. Do code summarization models process too much information? Function signature may be all what is needed. *ACM Transactions on Software Engineering and Methodology* 33, 6 (June 2024), Article 160, 35 pages. DOI: https://doi.org/10.1145/3652156

[14] Sidong Feng and Chunyang Chen. 2024. Prompting is all your need: Automated android bug replay with large language models. In *Proceedings ofthe IEEE/ACM 46th International Conference on Software Engineering (Lisbon, Portugal)*

(ICSE '24). Association for Computing Machinery, New York, NY, Article 67, 13 pages. DOI: https://doi.org/10.1145/3597503.3608137

[15] Qi Guo, Junming Cao, Xiaofei Xie, Shangqing Liu, Xiaohong Li, Bihuan Chen, and Xin Peng. 2024. Exploring the potential of ChatGPT in automated code refinement: An empirical study. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*, 1–13.

[16] Emilia Hansson and Oliwer Ellréus. 2023. Code correctness and quality in the era of AI code generation: Examining ChatGPT and GitHub Copilot. Master thesis. 69 pages. Retrieved from https://urn.kb.se/resolve?urn=urn:nbn:se:lnu:diva-121545

[17] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. 2020. CodeSearchNet challenge: Evaluating the state of semantic code search. arXiv:1909.09436. Retrieved from https://arxiv.org/abs/1909.09436

[18] Kailun Jin, Chung-Yu Wang, Hung Viet Pham, and Hadi Hemmati. 2024. Can ChatGPT support developers? An empirical evaluation of large language models for code generation In *MSR '24: Proceedings of the 21st International Conference on Mining Software Repositories*. Association for Computing Machinery, New York, NY, 167–171. DOI: https://doi.org/10.1145/3643991.3645074

[19] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large language models are zero-shot reasoners. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 35, 22199–22213.

[20] Brian Lester, Rami Al-Rfou, and Noah Constant. 2021. The power of scale for parameter-efficient prompt tuning. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP '21)*. Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih (Eds.), Association for Computational Linguistics, 3045–3059. DOI: https://doi.org/10.18653/v1/2021.emnlp-main.243

[21] Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023. Skcoder: A sketch-based approach for automatic code generation. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*. IEEE, 2124–2135.

[22] Jia Li, Yunfei Zhao, Yongmin Li, Ge Li, and Zhi Jin. 2023. Towards enhancing in-context learning for code generation. arXiv:2303.17780. Retrieved from https://arxiv.org/abs/2303.17780v1

[23] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. 2023. StarCoder: May the source be with you! arXiv:2305.06161. Retrieved from https://arxiv.org/abs/2305.06161

[24] Xiang Lisa Li and Percy Liang. 2021. Prefix-tuning: Optimizing continuous prompts for generation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL/IJCNLP '21)*. Chengqing Zong, Fei Xia, Wenjie Li, and Roberto Navigli (Eds.), Vol. 1, Association for Computational Linguistics, 4582–4597. DOI: https://doi.org/10.18653/v1/2021.acl-long.353

[25] Chao Liu, Xuanlin Bao, Hongyu Zhang, Neng Zhang, Haibo Hu, Xiaohong Zhang, and Meng Yan. 2023. Improving ChatGPT prompt for code generation. arXiv:2305.08360. Retrieved from https://arxiv.org/abs/2305.08360

[26] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Proceedings of the 37th Conference on Neural Information Processing Systems*, 21558–21572.

[27] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2024. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Proceedings of the Advances in Neural Information Processing Systems*, Vol. 36, 21558–21572

[28] Mingwei Liu, Tianyong Yang, Yiling Lou, Xueying Du, Ying Wang, and Xin Peng. 2023. CodeGen4Libs: A two-stage approach for library-oriented code generation. In *Proceedings of the 38th International Conference on Automated Software Engineering (ASE '23)*, 434–445. DOI: https://doi.org/10.1109/ASE56229.2023.00159

[29] Zhijie Liu, Yutian Tang, Xiapu Luo, Yuming Zhou, and Liang Feng Zhang. 2024. No need to lift a finger anymore? Assessing the quality of code generation by ChatGPT. *IEEE Transactions on Software Engineering* 50, 6 (2024), 1548–1584. DOI: https://doi.org/10.1109/TSE.2024.3392499

[30] Antonio Mastropaolo, Luca Pascarella, Emanuela Guglielmi, Matteo Ciniselli, Simone Scalabrino, Rocco Oliveto, and Gabriele Bavota. 2023. On the robustness of code generation techniques: An empirical study on GitHub copilot. In *Proceedings of the 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE '23)*. IEEE, 2149–2160.

[31] Nathalia Nascimento, Paulo Alencar, and Donald Cowan. 2023. Artificial intelligence vs. software engineers: An empirical study on performance and efficiency using ChatGPT. In *Proceedings of the 33rd Annual International Conference on Computer Science and Software Engineering*, 24–33.

[32] Phuong Thanh Nguyen, Juri Di Rocco, Davide Di Ruscio, and Massimiliano Di Penta. 2020. CrossRec: Supporting software developers by recommending third-party libraries. *Journal of Systems and Software* 161 (2020). DOI: https://doi.org/10.1016/j.jss.2019.110460

[33] Phuong Thanh Nguyen, Juri Di Rocco, Claudio Di Sipio, Davide Di Ruscio, and Massimiliano Di Penta. 2021. Recommending API function calls and code snippets to support software development. arXiv:2102.07508. Retrieved from https://arxiv.org/abs/2102.07508

[34] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2022. CodeGen: An open large language model for code with multi-turn program synthesis. In *Proceedings of the 11th International Conference on Learning Representations*. Retrieved from https://arxiv.org/abs/2203.13474

[35] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. BLEU: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, 311–318.

[36] Alec Radford, Jeff Wu, Rewon Child, David Luan, Dario Amodei, and Ilya Sutskever. 2019. Language Models Are Unsupervised Multitask Learners. Retrieved from https://api.semanticscholar.org/CorpusID:160025533

[37] Emily Reif, Daphne Ippolito, Ann Yuan, Andy Coenen, Chris Callison-Burch, and Jason Wei. 2022. A recipe for arbitrary text style transfer with large language models. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics*, Vol. 2. Association for Computational Linguistics, 837–848.

[38] Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. CodeBLEU: A method for automatic evaluation of code synthesis. arXiv:2009.10297. Retrieved from https://arxiv.org/abs/2009.10297

[39] Baptiste Roziere, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, et al. 2023. Code Llama: Open foundation models for code. arXiv:2308.12950. Retrieved from https://arxiv.org/abs/2308.12950

[40] Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, et al. 2022. Language models are multilingual chain-of-thought reasoners. In *Proceedings of the 11th International Conference on Learning Representations*. Retrieved from https://openreview.net/forum?id=fR3wGCk-IXp

[41] Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2023. Repository-level prompt generation for large language models of code. In *Proceedings of the International Conference on Machine Learning*. PMLR, 31693–31715.

[42] Mohammed Latif Siddiq, Joanna Cecilia Da Silva Santos, Ridwanul Hasan Tanvir, Noshin Ulfat, Fahmid Al Rifat, and Vinícius Carvalho Lopes. 2024. Using large language models to generate JUnit tests: An empirical study. In *Proceedings of the 28th International Conference on Evaluationand Assessment in Software Engineering (EASE '24)*. Association for Computing Machinery, New York, NY, 313–322. DOI: https://doi.org/10.1145/3661167.3661216

[43] Jiamou Sun, Zhenchang Xing, Rui Chu, Heilai Bai, Jinshui Wang, and Xin Peng. 2019. Know-how in programming tasks: From textual tutorials to task-oriented knowledge graph. In *Proceedings of the 2019 IEEE International Conference on Software Maintenance and Evolution (ICSME '19)*. IEEE, 257–268.

[44] Lewis Tunstall, Leandro Von Werra, and Thomas Wolf. 2022. *Natural Language Processing with Transformers*. O'Reilly Media, Inc.

[45] Priyan Vaithilingam, Tianyi Zhang, and Elena L. Glassman. 2022. Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models. In *Proceedings of the Chi Conference on Human Factors in Computing Systems Extended Abstracts*, 1–7.

[46] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 5998–6008.

[47] Ben Wang and Aran Komatsuzaki. 2021. GPT-J-6B: A 6 Billion Parameter Autoregressive Language Model. Retrieved from https://github.com/kingoflolz/mesh-transformer-jax

[48] Chaozheng Wang, Yuanhang Yang, Cuiyun Gao, Yun Peng, Hongyu Zhang, and Michael R. Lyu. 2022. No more fine-tuning? An experimental evaluation of prompt tuning in code intelligence. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 382–394.

[49] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the Neural Information Processing Systems (NeurIPS '22)*, 24824–24837. Retrieved from http://papers.nips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html

[50] Jules White, Sam Hays, Quchen Fu, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. ChatGPT prompt patterns for improving code quality, refactoring, requirements elicitation, and software design. arXiv:2303.07839. Retrieved from https://arxiv.org/abs/2303.07839

[51] Frank F. Xu, Uri Alon, Graham Neubig, and Vincent Josua Hellendoorn. 2022. A systematic evaluation of large language models of code. In *Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming*, 1–10.

[52] Imam Nur Bani Yusuf, Lingxiao Jiang, and David Lo. 2022. Accurate generation of trigger-action programs with domain-adapted sequence-to-sequence learning. In *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, 99–110.

[53] Daoguang Zan, Bei Chen, Zeqi Lin, Bei Guan, Wang Yongji, and Jian-Guang Lou. 2022. When language model meets private library. In *Findings of the Association for Computational Linguistics (EMNLP '22)*, 277–288.

[54] Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2023. CERT: Continual pre-training on sketches for library-oriented code generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI-22*. Lud De Raedt (Ed.), International Joint Conferences on Artificial Intelligence Organization, 2369–2375. DOI: https://doi.org/10.24963/ijcai.2022/329 Main Track

[55] Shuai Zhao, Jinming Wen, Luu Anh Tuan, Junbo Zhao, and Jie Fu. 2023. Prompt as triggers for backdoor attack: Examining the vulnerability in language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Houda Bouamor, Juan Pino, and Kalika Bali (Eds.), Association for Computational Linguistics, Singapore, 12303–12317. DOI: https://doi.org/10.18653/v1/2023.emnlp-main.757

[56] Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, Teng Su, Zhilin Yang, and JieTang. 2023. CodeGeeX: A pre-trained model for code generation with multilingual benchmarking on HumanEval-X. In *Proceedings of the 29thACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '23)*. Association for Computing Machinery, NewYork, NY, 5673–5684. DOI: https://doi.org/10.1145/3580305.3599790

Received 4 December 2023; revised 4 July 2024; accepted 19 August 2024